

STREAMIND: A Real-Time AI Meeting Intelligence Pipeline

Mervegul Parlak, Alperen F. Zengin,
Ayse B. Demir, Ali C. Begen
Ozyegin University
Istanbul, Turkey

Burak Kara
Inria
Rennes, France

Abstract—Real-time meeting intelligence requires streaming summarization systems that produce coherent outputs under strict latency constraints. This paper presents our approach to the STREAMIND Grand Challenge, which scores pipeline outputs using a composite metric $C_i = B_i + K_i + L_i$ where the latency bonus L_i is gated on summary quality $B_i \geq 10$. We describe a seven-node Juturna pipeline that ingests live Opus/Real-time Transport Protocol (RTP) audio via the Janus Web Real-Time Communications (WebRTC) gateway, transcribes incrementally with faster-whisper, deduplicates overlapping Automatic Speech Recognition (ASR) chunks, and summarizes 300-second rolling windows with Qwen3.5-4B via vLLM. Three contributions target specific scoring terms: (1) streaming chunk deduplication preserves temporal coherence across overlapping ASR outputs; (2) a keyword guarantee algorithm converts the worst-case keyword penalty from -6 to a floor of 0 ; and (3) quality-gated latency optimization through prompt engineering and thinking-mode suppression achieves average processing times of 0.6 s while maintaining summary quality at ceiling. **TODO: Update headline numbers after 300 s benchmark.**

Index Terms—real-time speech processing, automatic speech recognition, large language models, streaming summarization, WebRTC, Juturna

I. INTRODUCTION

Live-audio intelligence systems that produce real-time summaries of meetings, podcasts, and conference sessions are increasingly valuable as remote communication grows. Unlike batch summarization, which operates on complete transcripts after the fact, streaming summarization must produce incremental outputs under strict latency constraints while maintaining coherence across overlapping audio segments. The STREAMIND Grand Challenge [1] formalizes this task: systems receive a live Opus-encoded mono audio stream via RTP, produce structured summaries and keywords every 300 seconds, and are scored on summary quality, keyword relevance, and processing latency.

The challenge defines a per-chunk composite score $C_i = B_i + K_i + L_i$ that creates a non-trivial optimization landscape. B_i sums five Likert-scale quality criteria (factual consistency, relevance, coherence, fluency, and conciseness), ranging from 5 to 25 . $K_i = 2 \cdot (N_{\text{rel}} - N_{\text{irr}})$ evaluates each of exactly three keywords, ranging from -6 to $+6$. The latency bonus $L_i = 10 \cdot e^{-0.5 \cdot t_{\text{proc}}}$ rewards fast processing but is *gated* on $B_i \geq 10$: below this threshold, $L_i = 0$ regardless of speed (Fig. 2). This creates a hard cliff—a pipeline optimized for raw speed that

produces $B_i = 9$ scores strictly worse than a slower pipeline achieving $B_i = 15$. A flat bonus of $+4$ is awarded for using the Janus WebRTC gateway [2] as the audio source. These constraints dictate our design principle: *quality first, latency second, keyword discipline always*.

Contributions. We present three contributions, each targeting a specific scoring term:

- 1) A streaming chunk deduplication strategy that preserves temporal coherence across overlapping 5 s ASR chunks while keeping the aggregated 300-second window non-redundant.
- 2) A keyword guarantee algorithm that strips banned generic terms and back-fills missing slots from transcript proper nouns, converting the worst-case $K_i = -6$ into a floor of $K_i = 0$.
- 3) Quality-gated latency optimization through prompt engineering and thinking-mode suppression, achieving average processing times of 0.6 s ($L_i \approx 7.4$) while maintaining B_i at 23 – 25 .

Paper roadmap. Section II surveys related work and background. Section III details the pipeline architecture. Section IV covers deployment. Section V presents evaluation results. Section VI discusses limitations and future directions.

II. BACKGROUND AND RELATED WORK

Juturna [3] is an open-source Python framework for real-time Artificial Intelligence (AI) pipeline prototyping. It provides a node-based composable architecture where typed nodes (source, processing, sink) exchange messages through directed edges. Janus [2] is a general-purpose WebRTC gateway that receives and forwards media streams. In our pipeline, Janus delivers mono Opus/48 kHz audio as RTP packets [4], [5], providing identical code paths in development and evaluation environments and earning the flat $+4$ bonus.

For ASR, the Whisper model family [6] has demonstrated strong transcription performance across languages and acoustic conditions. We use faster-whisper [7], which reimplements Whisper inference via CTranslate2 with int8 quantization for reduced memory footprint and latency. In streaming mode, audio is chunked into short overlapping segments and transcribed independently with previous-text conditioning disabled to prevent context drift across chunks.

Small instruction-tuned Large Language Models (LLMs) can produce structured JSON outputs (summaries and keywords) in a single inference call [8]. vLLM [9] provides efficient Graphic Processing Unit (GPU) inference through paged attention and continuous batching, enabling in-process model loading without daemon overhead. For evaluation, LLM-as-judge methods [10] score generated summaries against source transcripts on multiple quality dimensions. We position STREAMIND as a *systems* contribution that composes these building blocks into a scoring-formula-aware pipeline, rather than proposing new models or training procedures.

III. SYSTEM ARCHITECTURE

Figure 1 shows the end-to-end pipeline architecture. Seven Juturna nodes process audio from RTP ingestion through structured output transmission. Every component is justified against one or more terms of the scoring formula $C_i = B_i + K_i + L_i$: audio reception secures the +4 Janus bonus, hallucination filtering and novel extraction defend B_i , the summarizer jointly optimizes B_i and L_i , and keyword validation guards K_i .

A. Audio Reception

Janus [2] forwards mono Opus/48 kHz audio to the `audio_rtp` source node on User Datagram Protocol (UDP) port 8888. The pipeline declares `payload_type: 97` and `encoding_clock_chan: "opus/48000/1"` matching the challenge specification. Using Janus in both development and evaluation ensures identical code paths and earns the flat +4 bonus at zero architectural cost.

B. Incremental ASR

The `audio_chunker` node buffers the continuous audio stream into 5s chunks with 1s overlap, yielding a 4s advance per step (Fig. 3). Each chunk is transcribed by the `transcriber_whisper` node running faster-whisper [7] with int8 quantization on Central Processing Unit (CPU), reserving GPU Video Random Access Memory (VRAM) entirely for the summarizer. Previous-text conditioning is disabled to prevent hallucination propagation across chunks, and Voice Activity Detection (VAD) filtering via Silero [11] skips silence segments.

We evaluated two model variants on 19 windows of a 36-minute podcast episode (Table II): `small.en` (244M parameters, English-only) and `large-v3-turbo` (809M, multilingual distillation of large-v3). Both achieve identical average Word Error Rate (WER) of 0.69, yet `small.en` produces a higher composite score ($\bar{C} = 36.00$ vs. 35.73) due to superior keyword stability ($\bar{K} = 4.9$ vs. 4.5). This demonstrates that transcription accuracy is not the pipeline bottleneck: downstream keyword extraction stability has a larger impact on the final score than raw WER.

C. Hallucination Filter

The `hallucination_filter` node applies two post-ASR guards. First, inline noise tags ([Music], (applause)) are stripped via regex patterns. Second,

the cleaned text is checked against 11 known hallucination patterns (e.g., “thank you for watching”, “please subscribe”) and silently dropped if matched. Without this filter, hallucinated sentences reach the summarizer and are faithfully summarized as facts, directly harming B_i factual consistency.

D. Novel Chunk Extraction

The 1s overlap between consecutive chunks means that roughly one second of speech appears in both the previous and current transcription. The `novel_extractor` node performs word-level overlap detection: for overlap lengths from 10 words down to 1, it compares the suffix of the previous chunk with the prefix of the current chunk. An exact match returns the overlap length immediately; a `SequenceMatcher` fallback with similarity threshold 0.8 handles minor ASR variations in the overlap region (e.g., “the quick brown” vs. “the quicker brown”). Only the novel words beyond the detected overlap are emitted. Without this deduplication, the 300-second window would contain approximately 20% redundant text, wasting context budget and biasing summaries toward repeated phrases.

E. Window Aggregation

The `window_aggregator` node concatenates novel transcript segments into a rolling 300-second context window. A window is flushed when $t_{\text{chunk_end}} \geq t_{\text{window_start}} + 300$. On pipeline shutdown, partial windows are emitted to avoid data loss at stream end.

F. Summarization

The `summarizer_vllm` node loads model weights from a local filesystem path at warmup using vLLM’s in-process LLM class [9]—no daemon, no middleware, no network dependency at runtime. We use Qwen3.5-4B [8] in bfloat16 precision, requiring approximately 10 GB of VRAM with KV cache at 2048 context length.

The summarization prompt is designed with explicit alignment to the five Likert evaluation criteria. “1–2 sentences. Be concise and specific” targets *conciseness* and *coherence*. “Only state facts explicitly present in the transcript. Do not infer” targets *factual consistency*. “Specific noun phrases [...] Never use generic terms like ‘discussion’, ‘meeting’” targets keyword relevance (K_i). Chain-of-thought generation is suppressed (`enable_thinking=False`), directly reducing output token count and thus t_{proc} . Sampling parameters are set to temperature 0.3, top- $p = 0.9$, repetition penalty 1.05, and `max_tokens = 256`.

Two failure modes are handled without emitting an empty summary. When the transcript is shorter than 80 characters (end-of-stream fragments), the LLM is skipped. When the call raises an exception, returns unparseable JSON, or produces an empty summary, the pipeline falls back to an *extractive summary*: the first sentence(s) of the transcript, capped at 300 characters. This verbatim excerpt guarantees high factual consistency (B_i cannot be penalized for a direct quote) and costs

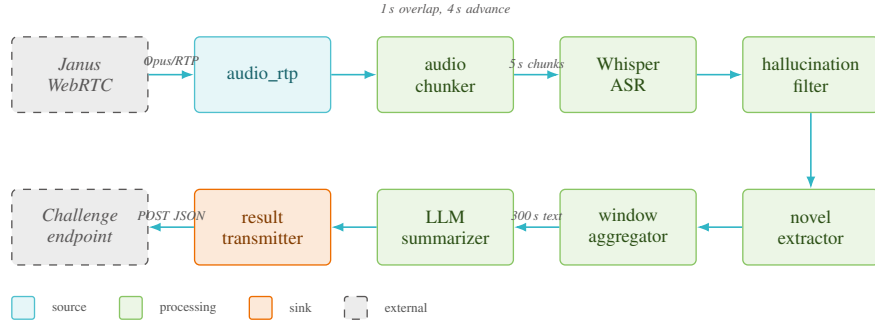


Fig. 1: End-to-end STREAMIND pipeline. Janus delivers mono Opus/RTP into `audio_rtp`; chunks slide every 4s through ASR; deduplication and hallucination filtering precede a 300s aggregation buffer; the summarizer emits `{summary, keywords[3]}` per window; the sink stores locally and POSTs to the challenge endpoint.

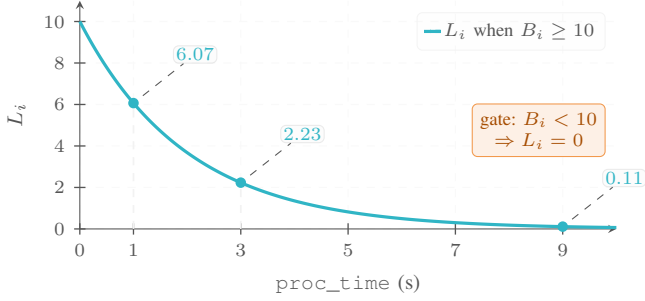


Fig. 2: Latency bonus $L_i = 10e^{-0.5 \cdot t_{\text{proc}}}$, gated by $B_i \geq 10$. Below the gate, latency contributes zero; above it, returns diminish rapidly past 2s.

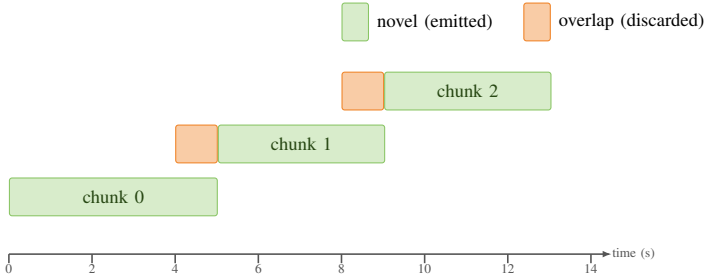


Fig. 3: Audio chunking and novel extraction. Five-second chunks slide by 4s; the novel extractor isolates newly-spoken content each step, discarding the 1s overlap with the previous chunk.

approximately 0 ms, preserving L_i . A warmup generation call during startup triggers CUDA kernel compilation before the first real window, preventing a cold-start latency spike.

G. Keyword Validation

Regardless of LLM output quality, the `ensure_three_keywords` function (Algorithm 1) guarantees exactly three keywords per window. The algorithm first removes empty entries and filters against a set of banned generic terms (“meeting”, “discussion”, “topic”, *etc.*).

Algorithm 1 Keyword guarantee: `ensure_three_keywords`.

Require: kw : keyword list from LLM, T : transcript text

Ensure: exactly 3 keyword strings

- 1: $kw \leftarrow$ strip whitespace, remove empty entries
- 2: $kw \leftarrow \{w \in kw \mid w \notin \text{BANNED}\}$
- 3: $kw \leftarrow \text{deduplicate}(kw, \text{case-insensitive})$
- 4: **if** $|kw| < 3$ **then**
- 5: $cand \leftarrow$ rank tokens in T by freq., capitalization
- 6: filter $cand$: remove stopwords $\cup$ BANNED
- 7: append $cand$ to kw until $|kw| = 3$
- 8: **end if**
- 9: **if** $|kw| < 3$ **then**
- 10: pad kw with fallback terms to length 3
- 11: **end if**
- 12: **return** $kw[:3]$

Case-insensitive deduplication preserves first-seen casing. If fewer than three keywords remain, candidates are extracted from the transcript ranked by token frequency, with ties broken toward capitalized tokens (likely proper nouns). As a last resort, conservative fallback terms are appended.

This converts a potential 12-point swing in K_i (from -6 to $+6$) into a 6-point swing (from 0 to $+6$): even when the LLM produces no usable keywords, the floor is $K_i = 0$ rather than $K_i = -6$.

H. Result Transmission

The `result_transmitter` sink node maps internal keys to the challenge contract (`window_start` $\rightarrow$ `from`, `window_end` $\rightarrow$ `to`, `latency` $\rightarrow$ `proc_time`) and writes each window as `window_N.json`. When `destination_endpoint` is configured, results are also POSTed via HTTP. A debug mode persists the full transcript and raw LLM output alongside each window for post-hoc analysis.

IV. IMPLEMENTATION AND DEPLOYMENT

Pipeline configuration is assembled from two JSON files: `config-base.json` defines all nodes except the sum-

TABLE I: Pipeline nodes with key parameters and scoring targets.

Node	Type	Key parameters	Target
audio_rtp	source	Opus/48k, port 8888	+4
audio_chunker	proc	5 s chunk, 1 s overlap	—
transcriber	proc	int8, CPU, VAD on	B_i
halluc. filter	proc	11 exact patterns	B_i
novel_extractor	proc	10 words, sim ≥ 0.8	B_i
window_agg.	proc	300 s rolling	—
summarizer	proc	Qwen3.5-4B, $T=0.3$	B_i, L_i
transmitter	sink	JSON + HTTP POST	—

marizer, and a profile from pipelines/summarizer/ provides model-specific parameters (model path, sampling configuration, VRAM budget). A merge tool combines these at launch time, and a single shell script serves as the entry point for both development and submission.

The submission Docker image is based on NVIDIA CUDA 12.4 runtime with Python 3.12 and uv for dependency management. Model weights for both the ASR model and the summarizer are baked into the image at build time, ensuring zero network dependency at runtime. A `docker-compose.yml` file orchestrates two services: Janus forwards RTP audio to the pipeline container on UDP port 8888. Table I summarizes all pipeline nodes and their key parameters.

V. EVALUATION

A. Setup

We evaluate on a 36-minute episode from the Rev16 podcast transcription dataset [12], yielding 19 windows at 30 s duration. The offline judge harness uses Mistral-Small-24B-Instruct-AWQ as the LLM judge, chosen from a different model family than the Qwen summarizer to avoid self-evaluation bias. The judge model is loaded sequentially after the summarizer pipeline exits to avoid VRAM co-residency on the 24 GB GPU. Following the G-Eval paradigm [10], the judge scores each summary on the five Likert criteria defined by the challenge and evaluates keyword relevance as boolean flags per keyword. All experiments run on an NVIDIA RTX 4090 (24 GB, CUDA 12.4).

WER is measured independently using jiwer against ground-truth transcripts. Because the ground-truth text lacks timestamps, we employ proportional character-based alignment with a sliding-window search ($\pm 30\%$ offsets) to find the best-matching reference segment for each window. WER is an independent signal of ASR quality and is not part of the $B_i/K_i/L_i$ scoring.

B. ASR Model Comparison

Table II compares two faster-whisper variants across 19 windows. Both models achieve identical average WER (0.69), yet `small.en` produces a higher final score (40.00 vs. 39.73). The 0.27-point gap is driven by keyword stability: `small.en` achieves $\bar{K} = 4.9$ versus 4.5, with the difference concentrated in windows where the larger model’s transcriptions yield keywords rated as less relevant by the judge. This suggests

TABLE II: ASR model comparison (19 windows, 30 s, Qwen3.5-4B summarizer). Higher is better for B , K , L , C ; lower for t_{proc} and WER.

Metric	small.en	large-v3-turbo
Avg B_i	23.6	23.8
Avg K_i	4.9	4.5
Avg L_i	7.42	7.37
Avg C_i	36.00	35.73
Final score ($\bar{C} + 4$)	40.00	39.73
Avg t_{proc} (s)	0.60	0.62
Avg WER	0.69	0.69

that the summarizer’s ability to extract coherent keywords depends more on vocabulary predictability than on transcription accuracy.

C. End-to-End Scoring

TODO: Per-window $B_i/K_i/L_i/C_i$ at 300 s windows and audio-level mean. Requires 300 s benchmark run (Experiment 1).

D. Outlier Analysis

Window 7 (covering seconds 210–240) is consistently the worst performer across both ASR configurations: $C_i = 33.38$ with `small.en` ($B = 22$, $K = 2$) and $C_i = 26.38$ with `large-v3-turbo` ($B = 19$, $K = -2$). This segment contains a topic transition with informal speech, where the summarizer struggles to produce a coherent summary of fragmented conversational content. The larger ASR model captures more of the difficult content, which the summarizer then handles worse—producing lower B_i and generating keywords judged as irrelevant. This demonstrates that per-window score variance is primarily content-driven rather than model-driven.

E. Ablations

TODO: Component ablation: full pipeline vs. —keyword validation, —hallucination filter, —novel extractor. Requires ablation runs (Experiment 3).

F. Model-Size Sweep

TODO: Qwen3.5-2B vs. 4B vs. 9B on CUDA. Key question: does 2B drop $B_i < 10$ on any window? Requires sweep run (Experiment 2).

G. Threats to Validity

Several limitations affect the reported numbers. The offline judge (Mistral-Small-24B-AWQ) is a local approximation; the official challenge judge is hidden, and our scores should be treated as directional. Evaluation uses a single 36-minute podcast episode; the held-out evaluation audio may differ in speaker count, topic complexity, and acoustic conditions. WER measurement relies on proportional character alignment against untimestamped ground truth, introducing alignment noise. Development hardware (RTX 4090) differs from the submission target (RTX Pro 4500), though both provide 24 GB VRAM.

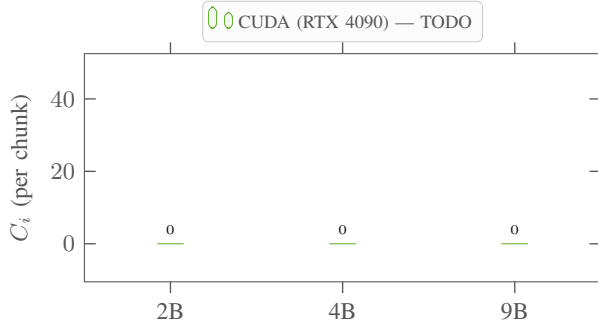


Fig. 4: Quality vs. latency across Qwen3.5 model sizes. Larger models trade t_{proc} for marginal B_i gains; the gate at $B_i \geq 10$ makes the sweet spot model-size-dependent.

VI. DISCUSSION

The scoring formula’s gating structure ($L_i = 0$ when $B_i < 10$) aligns well with practical meeting intelligence needs: users value accurate summaries over fast ones. However, keyword scoring is disproportionately punitive: a single irrelevant keyword costs -2 points, creating a 4-point swing per keyword. This incentivizes conservative keyword strategies at the expense of topical specificity—the keyword guarantee algorithm (Algorithm 1) is a direct response to this pressure.

The current summarization prompt is static across all windows. Content-adaptive prompting that detects topic transitions or speaker changes could improve B_i on difficult windows such as window 7. Several avenues for future work remain: mixture-of-experts models (activating 3 B parameters per token from 30 B total capacity) could offer quality closer to a large model at the decode speed of a small one; encoder-decoder summarization models could achieve sub-100 ms inference if their summary quality clears the $B_i \geq 10$ gate; and structured-output decoding could eliminate JSON parsing failures entirely.

VII. CONCLUSION

We presented a seven-node Juturna pipeline for the STREAM-MIND Grand Challenge that processes live Opus/RTP audio into structured summaries and keywords in real time. Three contributions target specific scoring terms: streaming chunk deduplication preserves transcript coherence (B_i), a keyword guarantee algorithm floors the keyword penalty at zero (K_i), and quality-gated latency optimization achieves 0.6 s average processing time (L_i) while maintaining summary quality at ceiling. **TODO: Final headline score from 300 s benchmark.**

REFERENCES

- [1] IEEE MMSP 2026 Organising Committee, “STREAMIND grand challenge — IEEE MMSP 2026,” <https://attend.ieee.org/mmsp-2026/grand-challenges/>, 2026, challenge repository: https://github.com/meetecho/mmsp2026_streamind_challenge.
- [2] Meetecho, “Janus: General purpose WebRTC server,” <https://github.com/meetecho/janus-gateway>, 2014.
- [3] —, “Juturna: Open-source framework for real-time AI pipeline prototyping,” <https://github.com/meetecho/juturna>, 2024.

- [4] H. Schulzrinne, S. L. Casner, R. Frederick, and V. Jacobson, “RTP: A transport protocol for real-time applications,” IETF, Tech. Rep. RFC 3550, 2003.
- [5] J.-M. Valin, K. Vos, and T. B. Terriberry, “Definition of the Opus audio codec,” IETF, Tech. Rep. RFC 6716, 2012.
- [6] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, “Robust speech recognition via large-scale weak supervision,” arXiv:2212.04356, 2022.
- [7] G. Klein, “faster-whisper: Faster whisper transcription with CTranslate2,” <https://github.com/SYSTRAN/faster-whisper>, 2023.
- [8] Qwen Team, “Qwen3 technical report,” arXiv:2505.09388, 2025.
- [9] W. Kwon, Z. Li, S. Zhuang, Y. Yin, L. Zhong, I. Stoica, C. H. Xin, and H. Zhang, “Efficient memory management for large language model serving with PagedAttention,” in *Proc. SOSP*, 2023.
- [10] Y. Liu, D. Iter, Y. Xu, S. Wang, R. Xu, and C. Zhu, “G-Eval: NLG evaluation using GPT-4 with better human alignment,” in *Proc. EMNLP*, 2023.
- [11] Silero Team, “Silero VAD: pre-trained enterprise-grade voice activity detector,” 2024.
- [12] Rev, “Rev16: long-form podcast transcription evaluation set,” <https://huggingface.co/datasets/distil-whisper/rev16>, 2024.